XML Metalanguage

The "father" of markup languages: XML

= EXtensible Markup Language

A simplified version of SGML

Originally created to overcome the limitations of HTML

The HTML language (especially until version 4):

- Cannot be modified to adapt to the structure of the specific content
- Is little structured

… but **XML goes far beyond** HTML: it **describes data**

Important!

XML (although the 'L' stands for 'Language') **is not a language**:

XML is a **METAlanguage!**

i.e., a language to define other languages (that is, a **set of rules** to define languages that conform to XML)

… consequently, saying "this document is written in XML" is technically wrong!

XML in seven points

(From `https://www.w3.org/XML/1999/XML-in-10-points-19990327`)

1. XML is a method for putting structured data in a text file
2. XML looks a bit like HTML but isn't HTML
3. XML is text, but isn't meant to be read
4. XML is a family of technologies
5. XML is verbose, but that is not a problem
6. XML is new, but not that new
7. XML is license-free, platform-independent and well-supported

An example

XML language to describe email messages

```
<?XML version="1.0" encoding="ISO-8859-1"?>
<message>
  <to>Luca</to>
  <from>Marco</from>
  <subject>Important information</subject>
  <body>
    Don’t forget our meeting tomorrow morning!
  </body>
</message>
```

XML…

XML allows the exchange of data between (sometimes “incompatible”) systems

It creates a common format to describe data

XML has no tags by itself

- Specific tags are created for specific XML languages, depending on their purpose
- New markup languages can be easily created (e.g., WML for the WAP protocol, SVG for vector graphics, etc.)

All opening tags must have their corresponding closing tags

No exceptions are allowed (apart from *empty-content* elements)

… XML…

XML tags are case-sensitive

`<message>` is different from `<Message>`

Spaces are not ignored

`one two three  four` is different from `one two three four`

Attribute values must always be in quotes

`<tag attribute=value>` wrong

`<tag attribute="value">` correct

`<tag attribute='value'>` correct

… XML…

E.g., description of a book (for a library)

```
<book>
   <title>Web Technologies</title>
   <classific id="inf-437" media= "paper" />
   <chapter>The HTML Language
     <paragraph>What is HTML?</paragraph>
     <paragraph>Structure of an HTML document</paragraph>
   </chapter>
   <chapter>Scripting languages
     <paragraph>Introduction to JavaScript</paragraph>
     <paragraph>Basic elements of the language</paragraph>
     <paragraph>Form validation</paragraph>
   </chapter>
 </book>
```

… XML…

XML tags are also called *elements*

An element can have any name

Provided that the name does not start with a digit, a punctuation character, “XML”, “Xml”, etc., and does not contain spaces

Elements can have different kinds of *content*

- *Element content* (e.g., `<book>` in the previous example)
- *Mixed content* (e.g., `<chapter>`)
- *Simple content* (e.g., `<paragraph>`)
- *Empty content* (e.g., `<classific>`)

Name conflicts

Tags with the same names may have different meanings in different languages/contexts

E.g.

```
<table>
  <tr>
    <td>Element 1</td>
    <td>Element 2</td>
  </tr>
</table>
```

```
<table>
  <name>
    Baroque table
  </name>
  <dim>100x200x120</dim>
</table>
```

Conflicts are solved with *namespaces*. Example:

```
<h:table xmlns:h="https://www.w3.org/TR/html4/">
  <h:tr>
    ...
```

```
<f:table xmlns:f="https://www.tables.com">
  <f:name>
    ...
```

XML: elements vs. attributes

When to use elements, and when attributes?

- If a property can assume more than one value → always use an element
- Rules of thumb for other cases:
  - (1) Use elements for "important" properties and attributes for "accessory" properties
  - (2) Use elements for properties that can assume "long" values

Examples (three possible descriptions for the entity *person*):

```
<person>
  <gender>male</gender>
  <name>Giuseppe</name>
  <surname>Verdi</surname>
</person>
```

```
<person gender="male">
  <name>Giuseppe</name>
  <surname>Verdi</surname>
</person>
```

```
<person gender="male" name="Giuseppe" surname="Verdi" />
```

XML: document validation

Well-formed documents

Documents complying with the syntactic rules of XML

Valid documents

Well-formed documents complying with the rules of a *Document Type Definition* or an *XML Schema*

Document Type Definition (DTD)

Document that defines “what is correct and what is not correct” in an XML document

XML Schema

More sophisticated alternative to DTD, based on an XML syntax

XML: Document Type Definition…

The DTD can be directly included in the XML document or (better solution) defined in an external file

Example:

```
<?xml version="1.0"?>
<!DOCTYPE message SYSTEM "mess.dtd">
<message>
  <to>Luca</to>
  <from>Marco</from>
  <subject>
    Un'informazione importante
  </subject>
  <body>Ricordati…</body>
</message>
```

mess.dtd

```
<!ELEMENT message (to,from,subject,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT subject (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

…XML: Document Type Definition…

Element declaration

- **<!ELEMENT** *element-name element-category***>**, if the element is "simple", i.e. it does not contain other elements
- **<!ELEMENT** *element-name* **(***element-content***) >**, if the element contains other elements
- **<!ELEMENT** *element-name* **EMPTY>**, if the element is empty (e.g., **
**)
- **<!ELEMENT** *element-name* **ANY>**, if any content is allowed

Data types

- PCDATA = Parsed Character Data (text that will be analyzed by a *parser*)
- CDATA = Character Data (text that will not be analyzed by a parser)

…XML: Document Type Definition…

Occurrences of child elements

Example:

- `<!ELEMENT book (title)>` ➔ one occurrence of `<title>`
- `<!ELEMENT book (title+)>` ➔ one or more occurrences of `<title>`
- `<!ELEMENT book (title*)>` ➔ zero or more occurrences of `<title>`
- `<!ELEMENT book (title?)>` ➔ zero or one occurrence of `<title>`
- `<!ELEMENT book (title,(chapter|paragraph))>` ➔
  one occurrence of `<title>` and one of `<chapter>` or `<paragraph>`
- `<!ELEMENT book (#PCDATA|title|chapter)*>` ➔
  generic (parsed) text and any number (also zero) of `<title>` and `<chapter>` elements

…XML: Document Type Definition…

Attribute declaration

`<!ATTLIST` *element-name attribute-name attribute-type default-value*`>`

Example:

```
<!ATTLIST classific id CDATA>
<!ATTLIST classific media (paper|CD) "paper">
```

Attribute types

E.g., `CDATA`, `(t1|t2|…|tn)`, `ENTITY`, …

Attribute default values

E.g., a string, `#REQUIRED`, `#IMPLIED`, …

…XML: Document Type Definition…

Entities to define replacement values

- Parsed Internal General Entity Declaration

  `<!ENTITY` *entityName* "*entityValue*"`>`

  E.g., in the DTD: `<!ENTITY CompanyName 'WWW Company Ltd'>`

  in the XML file: `<Vendor>&CompanyName;</Vendor>`

- Parsed External General Entity Declaration: the value for the replacement is read from an external file (parsed as XML data)

  `<!ENTITY` *entityName* `SYSTEM "`*entityUri*`">`

  E.g., in the DTD: `<!ENTITY BookChapter1 SYSTEM "Chapter1.xml">`

  in the XML file: `<book>&BookChapter1;</book>`

…XML: Document Type Definition

- Parameter entities: create reusable sections of replacement text

  <!ENTITY % *name definition*>

  <!ENTITY % *name* **SYSTEM** *uri*>

  <!ENTITY % *name* **PUBLIC** *formalPublicIdentifier uri*>

  E.g., in the DTD: **<!ENTITY % area "name, street, pincode, city">**

  <!ENTITY % contact "phone">

  <!ELEMENT residence (%area;, %contact;)>

  A formal public identifier uniquely identifies a product, specification, or document and has the structure **field 1//field 2//field 3//field 4**

  E.g., **-//W3C//DTD XHTML 1.0 Transitional//EN**

XML Schema as a (good) alternative to DTD

Example:

```
<?xml version="1.0"?>
<note>
  <to>Luca</to>
  <from>Marco</from>
  <heading>Reminder</heading>
  <body>Don't forget me this
   weekend!</body>
</note>
```

DTD

```
<!ELEMENT note
 (to, from, heading, body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

XML Schema

```
<?xml version="1.0"?>
<xs:schema  xmlns:xs="https://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.w3schools.com"
xmlns="http://www.w3schools.com"
elementFormDefault="qualified">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

XSL…

XSL = eXtensible Stylesheet Language

Special “stylesheet” which defines how an ordinary web browser must display an XML document

XSLT = XSL Transformation

XSL portion used to **translate** an XML document written in a certain language into another language (e.g., XHTML… but the destination language needs not to be an XML language)

Is it supported by web browsers?

Yes: all browsers support XSLT

… XSL – An example

```
<?xml version="1.0"
  encoding="ISO-8859-1"?>
<catalog>
  <cd>
    <title>Symphony n. 9</title>
    <artist>Beethoven, L.</artist>
    <country>Germany</country>
    <company>Deutsche Gr.</company>
    <price>10.90</price>
    <year>1987</year>
  </cd>
  ...
  ...
</catalog>
```

XML document

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
 xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <html>
      ...
      <body>
        <h1>My CD collection</h1>
        <table>
          <tr>
            <th>Title</th>
            <th>Artist</th>
          </tr>
          <xsl:for-each select="catalog/cd">
            <tr>
              <td><xsl:value-of select="title"/></td>
              <td><xsl:value-of select="artist"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

XSL document

Languages based on XML - *XHTML…*

XHTML is HTML complying with XML rules

- Documents must be well-formed
- Elements must be nested correctly
- Tags must be lowercase
- Attribute values must be written within quotes
- The **`name`** attribute is substituted by the **`id`** attribute (with the exception of the **`<input>`** element)
- All attributes must have a value

  E.g., **`<td nowrap="nowrap">`** instead of **`<td nowrap>`**
- All tags must have a closing tag

  With the syntax **`<tag />`** for empty elements

  E.g., **`<img src= "myimage.jpg" />`**

… XHTML

XHTML can use different DTDs

The `!DOCTYPE` declaration is mandatory

- XHTML 1.0 Strict

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
 "https://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

- XHTML 1.0 Transitional

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

- XHTML 1.0 Frameset (should not be used, because frames should be avoided!)

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

Languages based on XML - *WML*

WML = Wireless Markup Language

Was the language used to create WAP pages (WAP = Wireless Application Protocol)

A WML page is a *deck* composed of *cards*

Example:

```
<?xml version="1.0">
<!DOCTYPE wml PUBLIC "-//WAPFORUM//DTD WML 1.1//EN"
 "http//www.wapforum.org/DTD/wml_1.1.xml">
<wml>
  <card id="n1">
    <p>Hello everybody!</p>
  </card>
  <card id="n2">
    <p>Welcome to this WML example</p>
  </card>
</wml>
```

Languages based on XML - *SMIL*

SMIL = Synchronized Multimedia Integration Language

Allowed time coordination of different kinds of web media

Text, images, animations and videos are integrated into web pages, but remain separate entities

Example:

```
<smil>
  <head>
    <layout>
      <root-layout width="330" height="430" />
      <region id="title" top="5" left="5" width="220"
              height="172" background-color="white" />
      <region id="main" top="185" left="12" width="320"
              height="240" background-color="black" />
    </layout>
  </head>
  <body>
    <par> <!-in parallel... -->
      <seq repeat="20">
        <img src="im1.gif" region="title" dur="10s" />
        <img src="im2.gif" region="title" dur="10s" />
      </seq>
      <video src="esvideo.rm" region="main">
    </par>
  </body>
</smil>
```

Languages based on XML - *MathML*

MathML = Mathematical Markup Language

Allows the definition of math expressions

Example:

$$\{x \mid x \geq 0\} = [0, \infty)$$

```
...
<e>
  <eq />
  <set>
    <condition>
      <ci>x</ci>
      <e>
        <geq /><ci>x</ci><cn>0</cn>
      </e>
    </condition>
  </set>
  <interval closure="closed-open">
    <cn>0</cn>
    <ci>&infty;</ci>
  </interval>
</e>
...
```

Languages based on XML - *SVG*

SVG = Scalable Vector Graphics

Allows the description of static and dynamic bidimensional vector graphics

Example:


```
<!DOCTYPE html>
<html>
  <body>
    <svg width="400" height="180">
      <rect x="50" y="20" width="150"
       height="150" style="fill:blue;
       stroke:pink; stroke-width:5;
       fill-opacity:0.1; stroke-opacity:0.9" />
       Unfortunately your browser does not
       support SVG
    </svg>
  </body>
</html>
```